

IDS TEMELLERİ

[Kaynak](#)

İÇERİK

1. IDS (Intrusion Detection System) Nedir?
2. IDS Türleri: Dağıtım ve Tespit Yöntemleri
3. Snort: Dünyanın En Yaygın Açık Kaynak IDS Çözümü
4. Snort Kullanımı ve Kural Yazımı
5. Uygulamalı Lab: Snort ile PCAP Analizi ve Adli Bilişim (Forensics)

IDS (Intrusion Detection System) Nedir?

Bir ağın güvenliğini tasarlarken, tehditlerin sadece dışarıda tutulması değil, aynı zamanda içeri sızmayı başarmış olanların da tespit edilmesi gerekir.

Neden Sadece Firewall Yeterli Değil?

Bildiğimiz gibi **Firewall (Güvenlik Duvarı)**, genellikle bir ağın sınırına (boundary) yerleştirilen ilk savunma hattıdır. Temel amacı, ağa gelen (incoming) ve ağdan giden (outgoing) trafiği kontrol etmektir. Bir bağlantı kurulmak istendiğinde Firewall bu trafiği inceler ve eğer önceden belirlenmiş Firewall kurallarını ihlal ediyorsa bu bağlantıyı doğrudan reddeder (deny).

Ancak güvenlikte asıl sormamız gereken soru şudur: *"Ya kurallara uygun gibi görünen bir trafikle içeri sızarlarsa?"*

Eğer bir saldırgan, meşru görünen (legitimate-looking) bir bağlantı üzerinden Firewall'u başarılı bir şekilde atlatırsa (bypass) ve ağın içinde zararlı eylemler (malicious activities) gerçekleştirmeye başlarsa, Firewall artık bu aşamada işlevsiz kalır. Çünkü Firewall bağlantının kurulmasına bir kez izin vermiştir. İşte tam bu noktada, Firewall'u geçen ve halihazırda kurulmuş olan bağlantıların içerideki aktivitelerini eş zamanlı olarak tespit edebilecek bir iç ağ güvenlik çözümüne ihtiyacımız vardır. Bu çözüm **IDS (Intrusion Detection System - Saldırı Tespit Sistemi)** olarak adlandırılır.

Kapı Görevlisi ve Güvenlik Kamerası Analogisi

Firewall (Kapı Görevlisi): Binanın ana girişinde durur. İçeri giren ve çıkan insanları kontrol eder. Elindeki listeye (kurallara) uymayanları kapıdan içeri almaz. Ancak her zaman zeki bir saldırganın kapı görevlisini kandırıp içeri sızma ihtimali vardır.

IDS (Güvenlik Kameraları): Kapı görevlisinin gözünden kaçan biri içeri girdiğinde devreye giren sistemdir. Bina içinde bulunur, köşelerde sessizce oturur ve içerideki tüm hareketleri

izler. Eğer o kişi içeride şüpheli veya zararlı bir hareket yaparsa, kameralar bunu yakalar.

IDS Nasıl Çalışır ve Tespiti Nasıl Yapar?

IDS, ağın içinde konumlanır ve ağ trafiğini pasif bir şekilde, sürekli olarak dinler/izler. Tespit mekanizması iki temel kavrama dayanır:

1. **Signature-based Detection (İmza Tabanlı Tespit):** Bilinen zararlı yazılımların, exploit'lerin (sömürü kodları) veya saldırı türlerinin kendine has veri paket dizilimlerini, yani dijital izlerini (imzalarını) arar. Gelen trafik, sistemin veri tabanındaki bilinen saldırı imzalarıyla eşleştirilir.
2. **Anomaly-based Detection (Anomali Tabanlı Tespit):** Sistemin veya ağın normal davranış profilini (baseline) çıkarır. Bu normalden sapan, daha önce görülmemiş her türlü olağandışı (abnormal) trafiği tespit eder. (*Örneğin; gece saat 03:00'te veritabanı sunucusundan dışarıya beklenmedik şekilde GB'larca verinin çıkması bir anomalidir ve sıfırdan gün -zero day- saldırılarını yakalamakta etkilidir.*)

Kritik Ayrım (Asla Unutulmayacak Kural): IDS, ağın içine veya dışına doğru giden zararlı bir trafiği tespit ettiğinde **güvenlik yöneticileri için sadece bir uyarı (alert) üretir**. IDS, gelen bu tespitler üzerinde **herhangi bir aksiyon almaz, trafiği düşürmez (drop) veya engellemez**. IDS sadece bir gözlemcidir; tek görevi zararlı aktiviteyi yöneticilere zamanında bildirmektir. (*Trafiği hem tespit edip hem de otomatik olarak engelleyen sistemlere IDS değil, IPS - Intrusion Prevention System denir*).

IDS Türleri: Dağıtım ve Tespit Yöntemleri

IDS çözümlerini verimli kullanabilmek için onları iki ana kategoriye ayırarak incelememiz gerekir: **Deployment Modes** (Dağıtım Modları) ve **Detection Modes** (Tespit Modları). Bu ayrım, IDS'in ağın neresinde duracağını ve tehditleri nasıl "tanıyacağını" belirler.

1. Dağıtım Modları (Deployment Modes)

IDS'in fiziksel veya mantıksal olarak nereye kurulduğuna göre yapılan sınıflandırmadır.

Host Intrusion Detection System (HIDS)

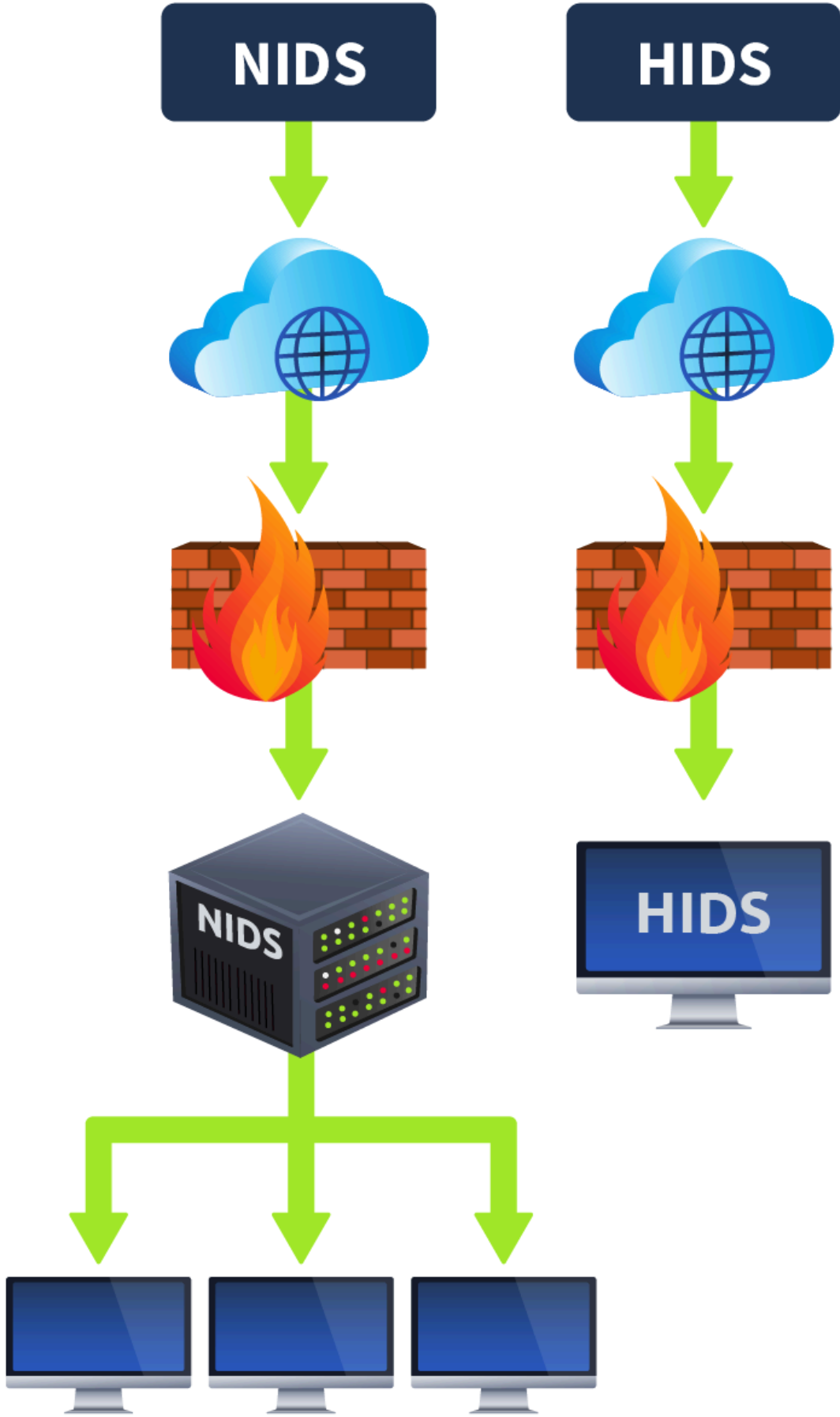
HIDS (Host Tabanlı Saldırı Tespit Sistemi), doğrudan korunmak istenen her bir sunucuya veya bilgisayara (host) ayrı ayrı kurulur.

- **Sorumluluk:** Sadece yüklü olduğu cihazdaki güvenlik tehditlerini izler.
- **Avantajı:** O cihazdaki dosya değişiklikleri, loglar ve kullanıcı aktiviteleri üzerinde çok detaylı bir görünürlük sağlar. Ağ trafiğinin şifreli olduğu durumlarda (uç noktada veri çözüldüğü için) içeride neler bittiğini en iyi HIDS görür.
- **Dezavantajı:** Büyük ağlarda yönetimi zordur. Her makineye tek tek kurulum ve yapılandırma gerektirir. Ayrıca cihazın kendi kaynaklarını (CPU, RAM) tükettiği için **resource-intensive** (kaynak yoğun) bir sistemdir.

Network Intrusion Detection System (NIDS)

NIDS (Ağ Tabanlı Saldırı Tespit Sistemi), belirli bir cihaza odaklanmak yerine tüm ağ trafiğini izlemek üzere ağın stratejik noktalarına yerleştirilir.

- **Sorumluluk:** Ağdaki tüm hostlar arasındaki trafiği dinleyerek şüpheli aktiviteleri yakalar.
- **Avantajı:** Tüm ağ için **centralized view** (merkezi bir görünüm) sunar. Tek bir noktadan tüm cihazları gözetleyebildiğimiz için yönetimi daha kolaydır.
- **Mantık:** NIDS, ağdaki trafiğin bir kopyasını alarak (mirroring) analiz eder, böylece tek tek cihazların performansını etkilemez.



2. Tespit Modları (Detection Modes)

IDS'in "neyin saldırı olduğunu" nasıl anladığına dair yöntemlerdir.

İmza Tabanlı IDS (Signature-Based IDS)

Bu yöntem, tıpkı bir antivirüs yazılımı gibi çalışır. Her saldırının kendine has bir "parmak izi" veya deseni vardır; buna **Signature** (İmza) denir.

- Çalışma Mantığı:** IDS, devasa bir imza veritabanına sahiptir. Gelen trafiği bu veritabanındaki desenlerle karşılaştırır. Eşleşme olursa alarm üretir.
- Güçlü Yanı:** Bilinen saldırıları çok hızlı ve düşük hata payı ile tespit eder. (Örn: **Snort** bu kategorideki en popüler araçtır).
- Zayıf Yanı - Zero-day Saldırıları:** **Zero-day** (Sıfırıncı gün) saldırılarını, yani daha önce hiç görülmemiş ve imzası veritabanına eklenmemiş saldırıları asla tespit edemez. Sadece "geçmişte yaşanmış ve kaydedilmiş" saldırıları tanır.

Anomali Tabanlı IDS (Anomaly-Based IDS)

İmzalara bakmak yerine "normal olmayan" davranışlara odaklanır.

- Çalışma Mantığı:** Önce ağın veya sistemin normal çalışma rutinini öğrenir. Buna **Baseline** (Temel Çizgi) denir. Eğer trafik bu normalin dışına çıkarsa (deviation), bunu bir saldırı olarak raporlar.
- Avantajı:** İmza gerektirmediği için **Zero-day** saldırılarını yakalama potansiyeli yüksektir.
- En Büyük Sorun - False Positives:** Çok fazla **False Positive** (Yanlış Pozitif - zararsız bir eylemin saldırı sanılması) üretir. Çünkü bazen meşru programlar da alışılmadık şekillerde davranabilir.
- Çözüm:** Bu hataları azaltmak için sistemin **fine-tuning** (ince ayar) ile sürekli optimize edilmesi ve normal davranışın manuel olarak çok iyi tanımlanması gerekir.

Hibrit IDS (Hybrid IDS)

Modern ve gelişmiş bir yaklaşımdır. Hem imza tabanlı hem de anomali tabanlı yöntemleri birleştirir.

- Mantık:** Bilinen tehditleri hızlıca imzalarla yakalar (hız kazandırır), bilinmeyen veya yeni nesil tehditleri ise anomali tespitiyle süzmeye çalışır. Böylece her iki yöntemin de avantajlarını kullanmış olur.

Özet Karşılaştırma Tablosu

Özellik	Signature-Based	Anomaly-Based
Hız	Çok Hızlı	Daha Yavaş (İşlem yükü fazla)
Zero-day Tespiti	Hayır	Evet
Hata Payı	Düşük (Doğru tespit)	Yüksek (Çok False Positive)
Kullanım Alanı	Küçük tehdit yüzeyleri, bilinen saldırılar	Modern ve karmaşık ağlar

Snort: Dünyanın En Yaygın Açık Kaynak IDS Çözümü

1998 yılında geliştirilen **Snort**, siber güvenlik dünyasının en köklü ve en yaygın kullanılan **open-source** (açık kaynak) IDS çözümüdür. Snort'un bu kadar popüler olmasının sebebi, hem **Signature-based** (imza tabanlı) hem de **Anomaly-based** (anomali tabanlı) tespit yeteneklerini tek potada eritmesidir.

Snort Kural Yapısı (Rules)

Snort, trafiği analiz ederken "kural dosyalarını" (rule files) temel alır. Bu dosyalar, trafiğin içinde neyin "kötü niyetli" olduğunu tanımlayan satırlardan oluşur.

- **Built-in Rules (Yerleşik Kurallar):** Snort yüklendiğinde içinde binlerce hazır saldırı desenini barındıran kural dosyalarıyla gelir. Bu kurallar, yaygın saldırıları (SQL Injection, Port Tarama vb.) tespit etmek için yeterlidir.
- **Custom Rules (Özel Kurallar):** Varsayılan kuralların kapsamadığı, kendi ağınıza veya özel bir trafiğe yönelik tespitler yapmak istiyorsanız kendi kurallarınızı yazabilirsiniz.
- **Kural Yönetimi:** Eğer sisteminizde sürekli yanlış alarm üreten (false positive) veya sizin için tehdit oluşturmayan bir trafik için kural varsa, bu yerleşik kuralı **disable** edebilir (devre dışı bırakabilir) ve yerine kendi mantığınıza uygun kurallar tanımlayabilirsiniz.

Snort Çalışma Modları

Snort sadece bir "alarm üreticisi" değildir; ihtiyaca göre üç farklı modda çalıştırılabilir. Her modun kullanım senaryosu farklıdır:



1. Packet Sniffer Mode (Paket Koklama Modu)

Bu modda Snort, ağ trafiğini sadece okur ve ekranda (konsolda) gösterir. Herhangi bir güvenlik analizi veya kural kontrolü yapmaz.

- **Kullanım Amacı:** Ağ izleme ve **troubleshooting** (hata giderme).
- **Senaryo:** Ağ ekibi bağlantı performansında bir sorun fark ettiğinde, trafiğin akışını detaylı görmek için bu modu kullanabilir. Çıktılar hem konsola basılabilir hem de bir dosyaya yazdırılabilir.

2. Packet Logging Mode (Paket Günlükleme Modu)

Ağ trafiğinin gerçek zamanlı olarak analiz edilip tespit edilenlerin kaydedildiği moddur. En büyük farkı, veriyi **PCAP** (standart paket yakalama formatı) olarak kaydetmesidir.

- **Kullanım Amacı:** Olay sonrası analiz ve **forensics** (adli bilişim) çalışmaları.
- **Senaryo:** Bir saldırı gerçekleştikten sonra, saldırganın hangi adımları izlediğini anlamak için (root cause analysis - kök neden analizi) geçmişe dönük trafik loglarına ihtiyaç duyulur. Bu modda tutulan PCAP dosyaları adli bilişim uzmanları için temel kanıt niteliğindedir.

3. Network Intrusion Detection System (NIDS) Mode

Snort'un "esas" modudur. Trafiği anlık olarak izler, elindeki kural dosyalarıyla (signatures) karşılaştırır ve bir eşleşme yakalarsa anında **Alert** (uyarı) üretir.

- **Kullanım Amacı:** Proaktif güvenlik izleme ve tehdit tespiti.
- **Senaryo:** Güvenlik ekibi (SOC - Security Operations Center), ağa sızma girişimlerini anında fark edip müdahale etmek istiyorsa Snort'u bu modda çalıştırır. Bir IDS çözümünden beklenen temel işlev budur.

Önemli Not: Snort'u bir IDS olarak kullanıyorsak en kritik modumuz **NIDS Mode**'dur. Ancak hata giderme yaparken Sniffer, kanıt toplarken ise Logging modları hayat kurtarır. Bir sonraki aşamada bu kuralların nasıl yazıldığını ve Snort'un arka plandaki mantığını inceleyeceğiz.

Snort Kullanımı ve Kural Yazımı

Snort'u verimli bir şekilde kullanabilmek ve ağdaki tehditleri yakalayabilmek için kurulum mimarisini, izin yapısını ve kural (rule) yazım sentaksını çok iyi anlamamız gerekiyor. Bu notlarda Snort'un kalbine iniyor ve kendi tespit kurallarımızı nasıl yazıp test edeceğimizi adım adım inceliyoruz.

Ağ Arayüzü ve "Promiscuous Mode" Mantığı (Kritik Detay)

Snort kurulumu sırasında bizden dinleyeceğimiz **network interface** (ağ arayüzü) ve ağ aralığı (range) istenir.

Normal şartlarda bir ağ kartı (NIC), sadece kendi IP adresine veya yayın (broadcast) adresine gelen paketleri yakalar, geri kalanları görmezden gelir. Eğer Snort'u sadece kendi bilgisayarımıza gelen saldırıları tespit etmek için çalıştırıyorsak normal mod yeterlidir. Ancak amacımız **tüm ağdaki** (başka cihazlara giden dahil) trafiği dinlemek ve tespit yapmaksa (yani Snort'u gerçek bir NIDS olarak konumlandırıyorsak), ağ arayüzümüzü kesinlikle **Promiscuous Mode**'a (Karma Mod) almalıyız. Bu mod, ağ kartına "hedefi ne olursa olsun, kablodan/havadan geçen tüm paketleri yakala ve içeri al" emrini verir.

Snort Dizin Yapısı ve Temel Dosyalar

Snort'un beyni ve sinir sistemi `/etc/snort` dizininde bulunur. Kurallar, konfigürasyonlar ve haritalamalar buradadır. Sistemin içini görmek için dizini listeliyoruz:

Bash

```
ubuntu@tryhackme:~$ ls /etc/snort
```

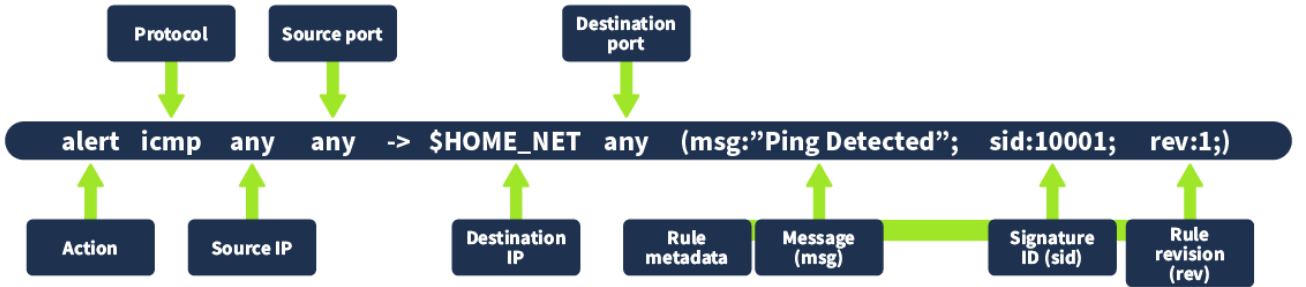
```
classification.config  reference.config  snort.debian.conf
community-sid-msg.map rules            threshold.conf
gen-msg.map           snort.conf       unicode.map
```

Burada asla unutmamamız gereken iki kritik yapı var:

1. **snort.conf** : Snort'un ana yapılandırma dosyasıdır. Hangi ağ aralığını dinleyeceğimiz, logların nereye yazılacağı ve *hangi kural dosyalarının aktif edileceği* hep bu dosyanın içinde tanımlanır.
2. **rules/ klasörü**: Snort'un tespit için kullandığı imza dosyalarının (kural setlerinin) bulunduğu klasördür.

Snort Kural Formatının Anatomisi (Rule Format)

Snort'ta bir kural yazarken veya okurken belirli bir sözdizimine (syntax) uymak zorundayız. Kural mantığını anlamak, Snort'u yönetmenin %80'idir.



Örnek bir senaryo üzerinden gidelim: Herhangi bir IP'den bizim ağımıza doğru gelen bir "Ping" (ICMP paketi) isteğini tespit etmek istiyoruz.

Örnek Kural: `alert icmp any any -> $HOME_NET any (msg:'Ping Detected'; sid:10001; rev:1;)`

Bu kuralı parçalarına ayırdığımızda şu mantık zinciri ortaya çıkar:

- **Action (Eylem)** - `alert` : Kural eşleştiğinde Snort'un ne yapacağını söyler. Burada `alert` diyerek sadece uyarı üretmesini istiyoruz. (Trafığı düşürmek isteseydik IPS modunda `drop` kullanabilirdik).
- **Protocol (Protokol)** - `icmp` : Kuralın hangi ağ protokolü için geçerli olduğunu belirtir. Ping işlemi ICMP kullandığı için bunu seçtik. (Diğer yaygın seçenekler: `tcp` , `udp` , `ip`).
- **Source IP (Kaynak IP)** - `any` : Trafığın nereden geldiğini belirtir. `any` diyerek internetteki veya ağdaki "herhangi bir IP'den" gelebileceğini söylüyoruz.

- **Source Port (Kaynak Port)** - `any` : Trafiğin çıktığı port. ICMP portsuz bir protokoldür ancak genel kural yapısını bozmamak adına veya TCP/UDP kurallarında "herhangi bir porttan çıkabilir" demek için `any` kullanırız.
- **Direction (Yön)** - `->` : Trafiğin akış yönünü gösterir. Kaynaktan hedefe doğru bir akış aradığımızı belirtir.
- **Destination IP (Hedef IP)** - `$HOME_NET` : Trafiğin kime geldiğini belirtir. Burada IP yazmak yerine `$HOME_NET` değişkenini kullandık. Bu değişkenin değeri (örneğin `192.168.1.0/24` gibi ağ aralığımız), `snort.conf` dosyasında önceden tanımlanmıştır.
- **Destination Port (Hedef Port)** - `any` : Hedefteki hangi porta geldiği. Yine `any` diyerek tüm portları/hedefleri kapsıyoruz.
- **Rule Metadata (Kural Meta Verisi)** - `(...)` : Parantez içindeki kısım. Kuralın kimliği ve ekrandaki çıktısıdır:
 - **msg (Message)**: Kural tetiklendiğinde ekranda veya logda göreceğimiz metindir. (Örn: "Ping Detected").
 - **sid (Signature ID)**: Kuralın benzersiz kimlik numarasıdır. **Kritik**: Kendi yazdığımız özel (custom) kuralların `sid` değeri mutlaka **1.000.000'dan (1 milyon) büyük** veya ayrılmış bloklarda olmalıdır ki yerleşik kurallarla çakışmasın.
 - **rev (Revision)**: Kuralın sürüm numarasıdır. Kuralı her güncellediğimizde bu sayıyı bir artırırız (rev:2, rev:3 vb.) ki hangi versiyonda olduğumuzu takip edebilelim.

Özel Kural Oluşturma (Rule Creation)

Şimdi teoriyi pratiğe dökelim. Özel (custom) kurallarımızı varsayılan kural dosyalarına değil, özellikle bu iş için ayrılmış olan `local.rules` dosyasına yazarız.

Önce dosyayı yetkili olarak bir metin editörüyle açıyoruz:

Bash

```
ubuntu@tryhackme:~$ sudo nano /etc/snort/rules/local.rules
```

Dosyanın sonuna, kendi yerel makinemize (loopback - 127.0.0.1) atılan pingleri yakalayacak şu kuralı ekliyoruz:

Plaintext

```
alert icmp any any -> 127.0.0.1 any (msg:"Loopback Ping Detected";  
sid:1000001; rev:1;)
```

Not: Verilen ham metinde sid:10003 kullanılmış olsa da, Snort standartlarında custom kurallar 1 milyondan başlar, ben standarda uygun şekilde 1000001 olarak düzeltiyorum ki gerçek hayatta hata almayalım.

Kuralı yazdıktan sonra `Ctrl+X`, `Y` ve `Enter` ile dosyayı kaydedip çıkıyoruz.

Kuralı Test Etme: Snort'u Canlı Trafikte Çalıştırma

Kuralımızı yazdık, şimdi Snort'u çalıştırıp test etmemiz lazım. Snort'u NIDS (Ağ Saldırı Tespit Sistemi) modunda başlatmak için detaylı bir komut kullanıyoruz.

Çalıştırma Komutu:

Bash

```
ubuntu@tryhackme:~$ sudo snort -q -l /var/log/snort -i lo -A console -c /etc/snort/snort.conf
```

Komutun Parametre Parametre İncelenmesi (Neden bunları yazdık?):

- **sudo** : Ağ arayüzlerini dinlemek (sniffing) root yetkisi gerektirir.
- **-q (quiet)**: Sessiz mod. Snort başlarken ekrana bastığı o uzun versiyon bilgilerini ve gereksiz banner yazılarını (fluff) gizler. Sadece önemli uyarıları görürüz.
- **-l /var/log/snort (log dir)**: Yakalanan paketlerin ve alertlerin loglanacağı dizini belirtir.
- **-i lo (interface)**: Dinleyeceğimiz ağ arayüzünü (NIC) belirtir. Biz 127.0.0.1'i dinleyeceğimiz için "loopback" arayüzü olan lo 'yu seçtik. (Eğer ana ağı dinleseydik eth0 veya ens33 gibi bir isim olacaktı).
- **-A console (Alert mode)**: Uyarıları (alert) anlık olarak terminal ekranına (console) basmasını söyler. Bunu yazmazsak uyarıları sadece log dosyasına yazar, ekranda göremeyiz.
- **-c /etc/snort/snort.conf (config)**: En önemli parametre! Snort'un kullanacağı yapılandırma dosyasının yolunu gösteriyoruz. Snort bu dosyayı okuyacak ve içindeki ayarlara (ve eklediğimiz local.rules'a) göre çalışacak.

Test Aşaması: Snort yukarıdaki komutla çalışıp terminali kilitledikten (dinlemeye başladıktan) sonra, *yeni bir terminal sekmesi açıp* şu komutu giriyoruz:

Bash

```
ubuntu@tryhackme:~$ ping 127.0.0.1
```

Çıktı Analizi: Snort'un çalıştığı ilk terminale döndüğümüzde şu çıktıları görmeliyiz:

Snort'u canlı bir arayüz yerine, kaydedilmiş bir dosya üzerinden çalıştırmak için `-r (read)` parametresini kullanırız.

Komut:

Bash

```
ubuntu@tryhackme:~$ sudo snort -q -l /var/log/snort -r Task.pcap -A console -c /etc/snort/snort.conf
```

Mantıksal Değişim: Bir önceki komutta kullandığımız `-i lo` (arayüz dinleme) parametresini sildik ve yerine `-r Task.pcap` (dosya okuma) parametresini koyduk. Geri kalan her şey (sessiz mod, log dizini, konsola yazdırma ve config dosyası) aynı. Snort sanki trafik o an akıyormuş gibi dosyayı okuyacak ve kurallarla eşleşen bir şey bulursa ekrana basacaktır. *(Buradaki Task.pcap kısmını, analiz edeceğiniz dosyanın tam yoluyla değiştirmeyi unutmayın).*

Uygulamalı Lab: Snort ile PCAP Analizi ve Adli Bilişim (Forensics)

Bu bölümde, teorik bilgilerimizi pratik bir senaryo üzerinde uyguluyoruz. Bir şirketin ağına saldırı düzenlenmiş ve elimizde bu saldırı anına ait trafik kayıtlarının bulunduğu

Intro_to_IDS.pcap dosyası var. Bir adli bilişim (forensic) uzmanı olarak görevimiz, Snort'u kullanarak bu kayıtlar içinde neler olup bittiğini, saldırganın hangi izleri bıraktığını ortaya çıkarmak.

Adım 1: Çalışma Dizinine Geçiş ve Dosya Kontrolü

Analiz edeceğimiz dosya `/etc/snort` dizini altında bulunuyor. İlk işimiz terminalde bu dizine gitmek ve dosyanın orada olup olmadığını teyit etmek.

Bash

```
# Dizine geçiş yapıyoruz
ubuntu@tryhackme:~$ cd /etc/snort

# Dosyayı listeliyoruz
ubuntu@tryhackme:/etc/snort$ ls
classification.config  Intro_to_IDS.pcap  reference.config
snort.debian.conf
community-sid-msg.map  rules              snort.conf         threshold.conf
gen-msg.map            threshold.conf     unicode.map
```

Neden buradayız? Snort'un ana konfigürasyon dosyası (`snort.conf`) ve analiz edeceğimiz `.pcap` dosyası aynı dizinde olduğu için komutları yönetmek burada daha kolaydır.

Adım 2: Snort ile Analizi Başlatma

Kaydedilmiş trafiği analiz etmek için Snort'u **-r (read)** parametresiyle çalıştıracğıız. Bu işlem sırasında Snort, paketi sanki o an ağdan geçiyormuş gibi tek tek okuyacak ve elindeki kural setleriyle (imzalarla) karşılaştıracaktır.

Analiz Komutu:

Bash

```
ubuntu@tryhackme:/etc/snort$ sudo snort -q -l /var/log/snort -r  
Intro_to_IDS.pcap -A console -c /etc/snort/snort.conf
```

Komutun Parametre Analizi (Hangi Bayrak Ne İşe Yarar?):

- **sudo** : Snort'un sistem dosyalarına ve log dizinine erişimi için gereklidir.
- **-q (Quiet)**: Snort'un başlangıç banner'larını ve teknik özetlerini gizler. Sadece kural eşleşmelerini (alertleri) görmek için kritiktir.
- **-l /var/log/snort** : Tespit edilen olayların log dosyalarının yazılacağı klasörü belirtir.
- **-r Intro_to_IDS.pcap** : "Read" anlamına gelir. Canlı trafiği değil, belirtilen PCAP dosyasını analiz etmesini söyler.
- **-A console** : Tespit edilen saldırı uyarılarını anında terminal ekranına basar. Analizi yapan kişi olarak çıktıyı canlı izlememizi sağlar.
- **-c /etc/snort/snort.conf** : Snort'un hangi kuralları kullanarak analiz yapacağını belirten konfigürasyon dosyasını yükler.

Adım 3: Çıktıları Yorumlama (Analiz Süreci)

Komutu çalıştırdıktan sonra terminal ekranına birçok alert (uyarı) düşmeye başlayacaktır. Bu aşamada dikkat etmemiz gereken noktalar şunlardır:

```
ubuntu@tryhackme: /etc/snort
File Edit View Search Terminal Help
ubuntu@tryhackme: /etc/snort$ sudo snort -q -l /var/log/snort -r Intro_to_IDS.pcap -A console -c /etc/snort/snort.conf
07/18-12:52:59.337559 [**] [1:1000002:1] SSH Connection Detected [**] [Priority: 0] (TCP) 10.11.90.211:54334 -> 10.10.161.151:22
07/18-12:52:59.510662 [**] [1:1000002:1] SSH Connection Detected [**] [Priority: 0] (TCP) 10.11.90.211:54334 -> 10.10.161.151:22
07/18-12:52:59.510663 [**] [1:1000002:1] SSH Connection Detected [**] [Priority: 0] (TCP) 10.11.90.211:54334 -> 10.10.161.151:22
07/18-12:52:59.693619 [**] [1:1000002:1] SSH Connection Detected [**] [Priority: 0] (TCP) 10.11.90.211:54334 -> 10.10.161.151:22
07/18-12:52:59.696547 [**] [1:1000002:1] SSH Connection Detected [**] [Priority: 0] (TCP) 10.11.90.211:54334 -> 10.10.161.151:22
07/18-12:52:59.696584 [**] [1:1000002:1] SSH Connection Detected [**] [Priority: 0] (TCP) 10.11.90.211:54334 -> 10.10.161.151:22
07/18-12:52:59.867507 [**] [1:1000002:1] SSH Connection Detected [**] [Priority: 0] (TCP) 10.11.90.211:54334 -> 10.10.161.151:22
07/18-12:52:59.868435 [**] [1:1000002:1] SSH Connection Detected [**] [Priority: 0] (TCP) 10.11.90.211:54334 -> 10.10.161.151:22
07/18-12:53:00.058645 [**] [1:1000002:1] SSH Connection Detected [**] [Priority: 0] (TCP) 10.11.90.211:54334 -> 10.10.161.151:22
07/18-12:53:00.057313 [**] [1:1000002:1] SSH Connection Detected [**] [Priority: 0] (TCP) 10.11.90.211:54334 -> 10.10.161.151:22
07/18-12:53:00.272189 [**] [1:1000002:1] SSH Connection Detected [**] [Priority: 0] (TCP) 10.11.90.211:54334 -> 10.10.161.151:22
07/18-12:53:00.444487 [**] [1:1000002:1] SSH Connection Detected [**] [Priority: 0] (TCP) 10.11.90.211:54334 -> 10.10.161.151:22
07/18-12:53:00.444488 [**] [1:1000002:1] SSH Connection Detected [**] [Priority: 0] (TCP) 10.11.90.211:54334 -> 10.10.161.151:22
07/18-12:53:00.623967 [**] [1:1000002:1] SSH Connection Detected [**] [Priority: 0] (TCP) 10.11.90.211:54334 -> 10.10.161.151:22
07/18-12:53:00.647745 [**] [1:1000002:1] SSH Connection Detected [**] [Priority: 0] (TCP) 10.11.90.211:54334 -> 10.10.161.151:22
07/18-12:53:00.830895 [**] [1:1000002:1] SSH Connection Detected [**] [Priority: 0] (TCP) 10.11.90.211:54334 -> 10.10.161.151:22
07/18-12:53:00.830918 [**] [1:1000002:1] SSH Connection Detected [**] [Priority: 0] (TCP) 10.11.90.211:54334 -> 10.10.161.151:22
07/18-12:53:01.364435 [**] [1:1000002:1] SSH Connection Detected [**] [Priority: 0] (TCP) 10.11.90.211:54334 -> 10.10.161.151:22
07/18-12:53:01.366033 [**] [1:1000002:1] SSH Connection Detected [**] [Priority: 0] (TCP) 10.11.90.211:54334 -> 10.10.161.151:22
07/18-12:53:01.536660 [**] [1:1000002:1] SSH Connection Detected [**] [Priority: 0] (TCP) 10.11.90.211:54334 -> 10.10.161.151:22
07/18-12:53:01.536660 [**] [1:1000002:1] SSH Connection Detected [**] [Priority: 0] (TCP) 10.11.90.211:54334 -> 10.10.161.151:22
07/18-12:53:01.710516 [**] [1:1000002:1] SSH Connection Detected [**] [Priority: 0] (TCP) 10.11.90.211:54334 -> 10.10.161.151:22
07/18-12:53:01.710517 [**] [1:1000002:1] SSH Connection Detected [**] [Priority: 0] (TCP) 10.11.90.211:54334 -> 10.10.161.151:22
07/18-12:53:01.710548 [**] [1:1000002:1] SSH Connection Detected [**] [Priority: 0] (TCP) 10.11.90.211:54334 -> 10.10.161.151:22
07/18-12:53:01.763394 [**] [1:1000002:1] SSH Connection Detected [**] [Priority: 0] (TCP) 10.11.90.211:54334 -> 10.10.161.151:22
07/18-12:53:08.992981 [**] [1:1000002:1] SSH Connection Detected [**] [Priority: 0] (TCP) 10.11.90.211:54334 -> 10.10.161.151:22
07/18-12:53:09.167665 [**] [1:1000002:1] SSH Connection Detected [**] [Priority: 0] (TCP) 10.11.90.211:54334 -> 10.10.161.151:22
07/18-12:53:09.923610 [**] [1:1000002:1] SSH Connection Detected [**] [Priority: 0] (TCP) 10.11.90.211:54334 -> 10.10.161.151:22
07/18-12:53:10.097583 [**] [1:1000002:1] SSH Connection Detected [**] [Priority: 0] (TCP) 10.11.90.211:54334 -> 10.10.161.151:22
07/18-12:53:10.292052 [**] [1:1000002:1] SSH Connection Detected [**] [Priority: 0] (TCP) 10.11.90.211:54334 -> 10.10.161.151:22
07/18-12:53:10.396573 [**] [1:1000002:1] SSH Connection Detected [**] [Priority: 0] (TCP) 10.11.90.211:54334 -> 10.10.161.151:22
07/18-12:53:10.467526 [**] [1:1000002:1] SSH Connection Detected [**] [Priority: 0] (TCP) 10.11.90.211:54334 -> 10.10.161.151:22
07/18-12:53:10.571659 [**] [1:1000002:1] SSH Connection Detected [**] [Priority: 0] (TCP) 10.11.90.211:54334 -> 10.10.161.151:22
07/18-12:53:10.609756 [**] [1:1000002:1] SSH Connection Detected [**] [Priority: 0] (TCP) 10.11.90.211:54334 -> 10.10.161.151:22
07/18-12:53:10.783691 [**] [1:1000002:1] SSH Connection Detected [**] [Priority: 0] (TCP) 10.11.90.211:54334 -> 10.10.161.151:22
```

1. **Alert Mesajları (msg):** Saldırganın ne yapmaya çalıştığını anlamamızı sağlar. Örneğin "SQL Injection attempt", "NMAP Scan" veya önceki görevde yazdığımız "Ping Detected" gibi mesajlar görebiliriz.
2. **Kaynak ve Hedef IP'ler:** Saldırganın hangi IP adresinden geldiğini ve ağıımızdaki hangi sunucuyu/cihazı hedef aldığını gösterir.
3. **SID (Signature ID):** Hangi kuralın tetiklendiğini benzersiz numarasıyla belirtir. Eğer kendi yazdığımız bir kural tetiklenirse, burada 1000001 gibi bizim atadığımız numarayı görürüz.
4. **Protokol Bilgisi:** Saldırının TCP, UDP veya ICMP üzerinden mi yapıldığını anlamamızı sağlar.

Kritik Hatırlatma: Kendi Kurallarımızın Rolü

Eğer bir önceki görevde /etc/snort/rules/local.rules dosyasına özel kurallar eklediysek ve bu kurallar snort.conf içinde aktifse, Snort PCAP dosyasını analiz ederken **bu kuralları da kullanacaktır**. Yani saldırgan eğer bizim kural yazdığımız bir yöntemi (örneğin ICMP paketleri) kullandıysa, bu durumu anında yakalayacağız.

Not: Analiz bittikten sonra ekranda kaç tane paket işlendiği ve kaç tane alert (uyarı) üretildiği özet olarak görünecektir. Bu özet bilgi, saldırının yoğunluğunu anlamak için temel teşkil eder.